The Road to C: From Wires to Words



Wameedh SC



Agenda

01 What Is a Computer?

02 Why We Need Computers

03 The Binary Number System

04 Computer Anatomy

05 Machine Code to Assembly

06 What Is a Compiler?

07 Where We Find C Today

08 Showing Output: printf

09 Datatypes & Placeholders

10 Variables & const

11 Reading Input: scanf

12 Arithmetic Operators

Question

Who can tell me what a computer actually is?

Answer:

A machine that processes information according to instructions.

Every computer system performs: **Input** → **Processing** → **Output**

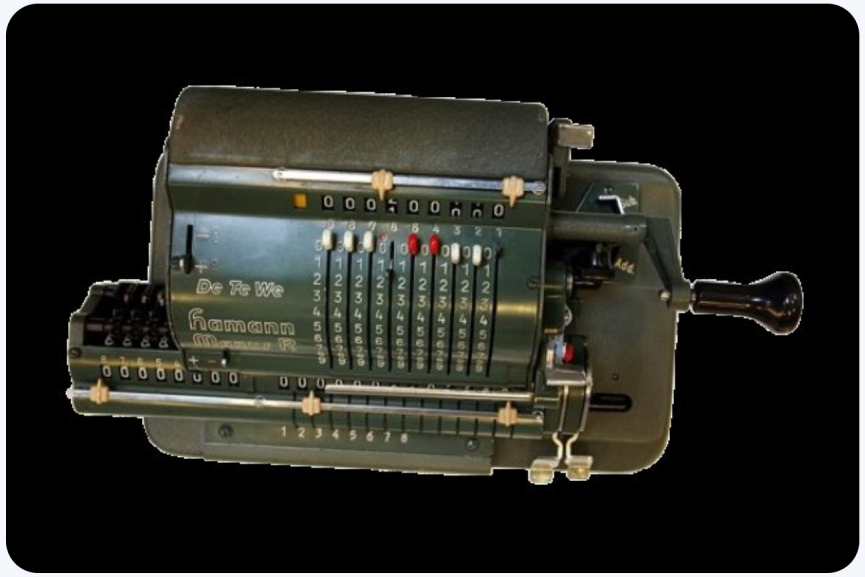


“Every computer, from your phone to a supercomputer, follows this same logic.”

Why We Need Computers?

Humans needed and created computers primarily to automate difficult, time-consuming, and error-prone tasks, particularly complex calculations, and to enhance human capabilities in managing information and solving problems.

During World War II, military needs drove early computer development — machines like Colossus and Bombe broke enemy codes, while ENIAC computed artillery trajectories.



Mechanical Computer

Question

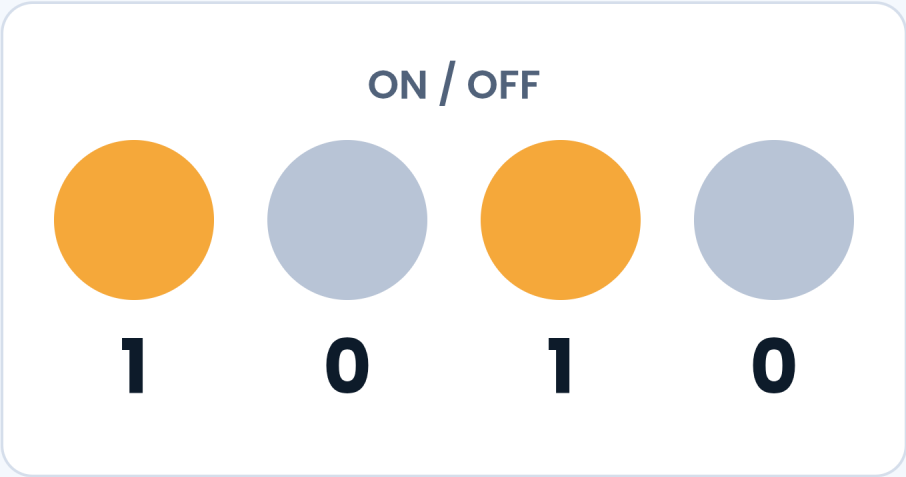
What is the binary number system?

Answer:

Humans use **base 10** → because of 10 fingers.

Machines use **base 2** → because of two stable states (ON/OFF).

Binary fits perfectly with electronic switches.



Examples (decimal → binary):

2 → 10	1 → 01	3 → 11	0 → 00

Information:

60

Babylonians: used base 60 (sexagesimal) — that's why we still have 60 seconds in a minute and 360 degrees in a circle.

20

Mayans: used base 20 (vigesimal) — likely because they counted on fingers and toes.

Let’s discover together computer anatomy



Question

What is a program?

From Machine Code to Assembly

Early programming = writing 0s and 1s.

Assembly introduced symbols easier for humans.

Both are low-level and hardware-specific.

Example :

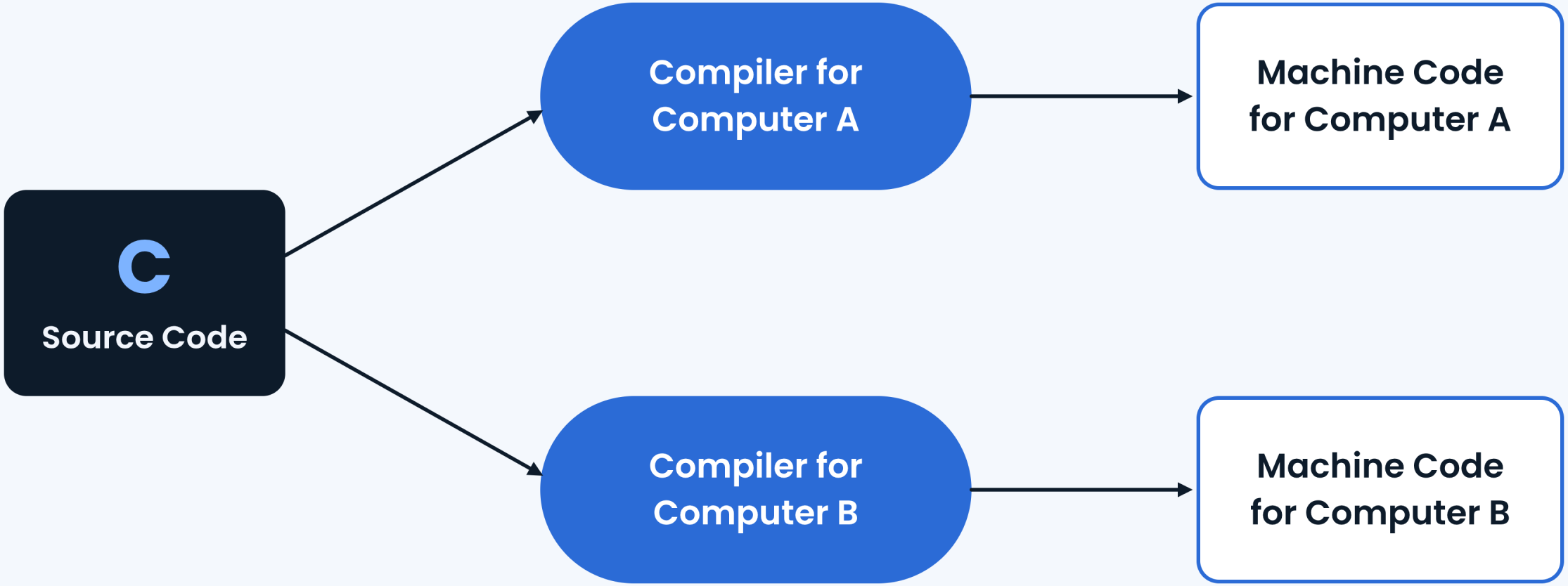
```
Machine : 10110000 00000001
Assembly : MOV AL, 01h
```



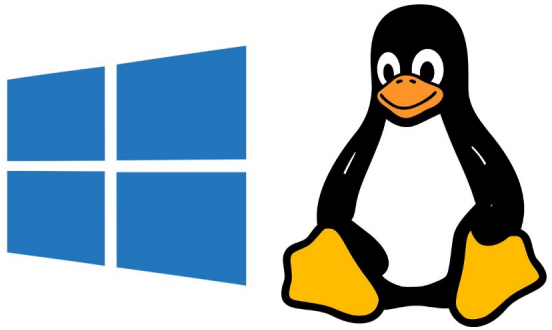


Question

What is a compiler?


C
Source Code
Compiler for Computer A
Compiler for Computer B
Machine Code for Computer A
Machine Code for Computer B

Where We Find C Today?



Operating systems

Windows and Linux kernel.



Embedded System

Cars, IoT, controllers

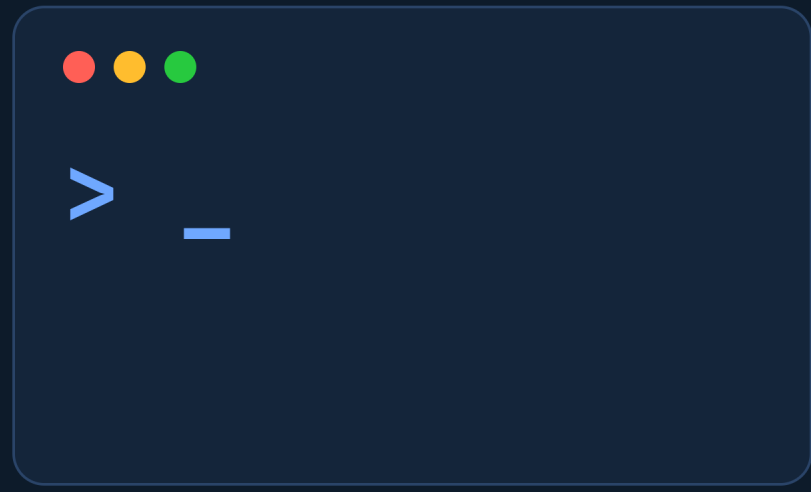


High-Performance Systems

Game engine, compilers, Databases

Let’s make a small demo

Don’t forget to download the C compiler on your machine (windows)



Question

How do we show something on the screen?

printf: Showing Output

- printf is a function from the **stdio.h** library. It prints text and values to the screen.
- Everything inside the quotes is printed exactly as written.
- The **\n** at the end moves the cursor to the next line.
- Every program that uses printf starts with: **#include <stdio.h>**

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Output

```
Hello, World!
```

Question

What kinds of data can a program store?

The Four Basic Datatypes

Every value in C has a datatype. It tells the computer what kind of value it is and how much memory it uses.

`int`

Whole numbers

```
42, -7, 0
```

Print with **%d**

usually 4 bytes

`float`

Decimal numbers

```
3.14, -0.5, 2.0
```

Print with **%f**

usually 4 bytes

`char`

One character

```
'A', 'z', '7'
```

Print with **%c**

usually 1 byte

`bool`

True or false

```
true, false
```

Print with **%d**

usually 1 byte

true prints as 1 and false prints as 0. bool needs #include <stdbool.h>. Sizes are typical and can vary.

printf: Placeholders with %

Use % followed by a letter to mark where a value will be printed. The values are written after the text, in the same order.

%d

int

Whole numbers

```
prints 42
```

%f

float

Decimal numbers

```
prints 3.14
```

%c

char

One character

```
prints A
```

%%

percent

Literal % sign

```
prints %
```

Tip: %.2f prints a float with exactly 2 digits after the dot. 3.14159 is printed as 3.14.

int and float: Numbers

`int`

Stores whole numbers. There is no decimal point.

```
printf("%d\n", 30);
printf("%d\n", -5);
```

- The decimal part is lost: 3.9 becomes 3 when stored as an int.
- Print with **%d**.

`float`

Stores numbers with a decimal point. It keeps about 7 digits of precision.

```
printf("%f\n", 19.99);
printf("%f\n", 3.14);
```

- Print with **%f**, or **%.2f** for 2 decimals.
- Use float when the value has a fraction.

char and bool

char

Stores one character, written in single quotes.

```
printf("%c\n", 'A');
printf("%c\n", '$');
```

- 'A' is a char, but "A" is text, not a char.
- Behind the scenes each char is a number. For example, A is 65.
- Print with **%c**.

bool

Stores true or false. It needs the stdbool.h library.

```
#include <stdbool.h>
printf("%d\n", true);
printf("%d\n", false);
```

- true is stored as 1 and false as 0.
- Print with **%d** to see 1 or 0.
- Used for yes or no questions, like "did the student pass?"

Question

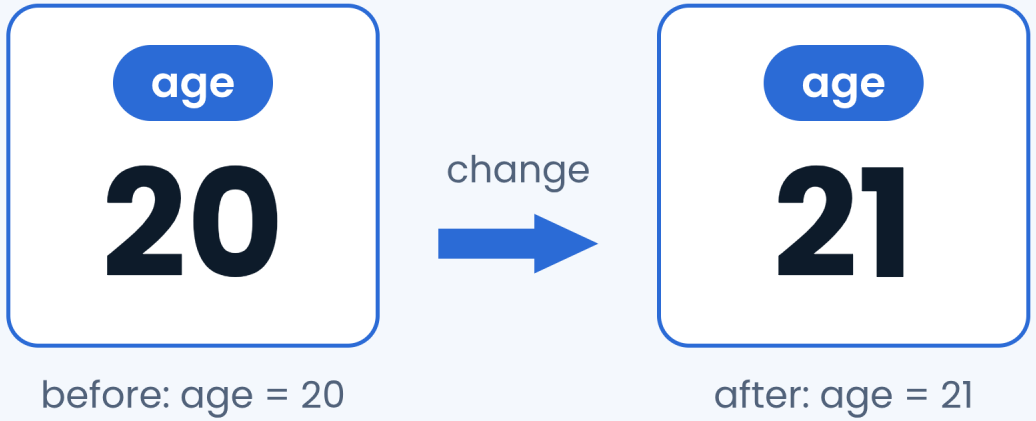
How do we keep a value to use later?

Variables: A Named Container

A variable is a named box in memory. It has two parts:

- **Name**: the label used to find the box, like **age**
- **Value**: what is stored inside the box, like **20**

The value can be changed at any time. The name stays the same.



The label stays. The content can change.

Declare, Assign and Change

Syntax

```
datatype name = value;
```

You can also declare first and give a value later: int age; then age = 20;

Naming rules

- Letters, digits and _ only
- Cannot start with a digit
- No spaces. Use student_age or studentAge
- Case-sensitive: Age and age are different
- Cannot be a keyword like int

```c
#include <stdio.h>
int main() {
    int age = 20;   // declare and assign
    printf("%d\n", age);   // prints 20
    age = 21;   // change the value
    printf("%d\n", age);   // prints 21
    return 0;
}
```

Output

```
20
21
```

printf: Mixing Text and Values

```c
#include <stdio.h>

int main() {
    int age = 20;
    float height = 1.75;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Height: %.2f\n", height);
    printf("Grade: %c\n", grade);
    return 0;
}
```

Output

```
Age: 20
Height: 1.75
Grade: A
```

- Each % is replaced by a value, in order.
- %d for int, %f for float and %c for char.
- %.2f limits a float to 2 digits after the dot.

const: A Value That Cannot Change

Adding **const** before the datatype makes a variable read-only. Once it has a value, it can never change.

- A const must get its value when it is created.
- If the program tries to change it, the compiler stops with an error.
- Use it for values that never change, like PI or a maximum score.

Normal variable

```
int score = 10;
score = 11;   // allowed
```

const variable

```
const int score = 10;
score = 11;   // error
```

const: Examples

```c
#include <stdio.h>

const float PI = 3.14;
const int MAX_STUDENTS = 30;

int main() {
    float radius = 2.0;
    float area = PI * radius * radius;
    printf("Area: %.2f\n", area);
    printf("Max: %d\n", MAX_STUDENTS);
    // MAX_STUDENTS = 40;  // ERROR: cannot change a const
    return 0;
}
```

Output

```
Area: 12.56
Max: 30
```

Why use const?

- Protects values from accidental changes.
- Easy to update in one place.
- Makes the code easier to read.

Question

How can the user give us a value?

scanf: Reading Input

scanf reads what the user types on the keyboard and stores it in a variable.

Syntax

```
scanf("format", &variable);
```

- The format works like printf: %d, %f or %c.
- The & (address of) tells scanf where to store the value.
- Needs #include <stdio.h>, like printf.

```c
#include <stdio.h>

int main() {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("You are %d years old\n", age);
    return 0;
}
```

Sample run

```
Enter your age: 20
You are 20 years old
```

The user types 20 and presses Enter.

scanf with Each Datatype

int
whole numbers

```
scanf("%d", &students);
```

Use **%d** for int values.

float
decimal numbers

```
scanf("%f", &price);
```

Use **%f** for float values.

char
one character

```
scanf(" %c", &grade);
```

The space before %c skips the Enter key left over from the last input.

bool
true or false

```
scanf("%d", &answer);
passed = answer;
```

scanf cannot store into a bool directly. Read a 1 or 0 into an int, then copy it into the bool.

Full Example

One program that reads all four datatypes. Each scanf stores its value in a variable.

- The space in " %c" skips the Enter key left over from the previous input.
- bool is read as an int, then copied into the bool.

Sample run

```
Age: 20
Height: 1.75
Grade: A
Passed (1 or 0): 1
20 1.75 A 1
```

```c
#include <stdio.h>
#include <stdbool.h>
int main() {
  int age;
  float height;
  char grade;
  int answer;
  bool passed;
  printf("Age: ");
  scanf("%d", &age);
  printf("Height: ");
  scanf("%f", &height);
  printf("Grade: ");
  scanf(" %c", &grade);
  printf("Passed (1 or 0): ");
  scanf("%d", &answer);
  passed = answer;
  printf("%d %.2f %c %d\n", age, height, grade, passed);
  return 0;
}
```

Question

What can we do with values?

Arithmetic Operators

Given a = 10 and b = 3

Operator	Name	Example	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division (whole numbers)	a / b	3
%	Remainder	a % b	1

```
printf("%d\n", a / b);   // 3
printf("%d\n", a % b);   // 1
```

Division of two int values drops the decimal part. 10 / 3 gives 3, not 3.33. For decimals, use a float: 10.0 / 3 gives 3.33.

Thank you for your attention!



Wameedh SC

